NumPy: Arrays & Vectorisation

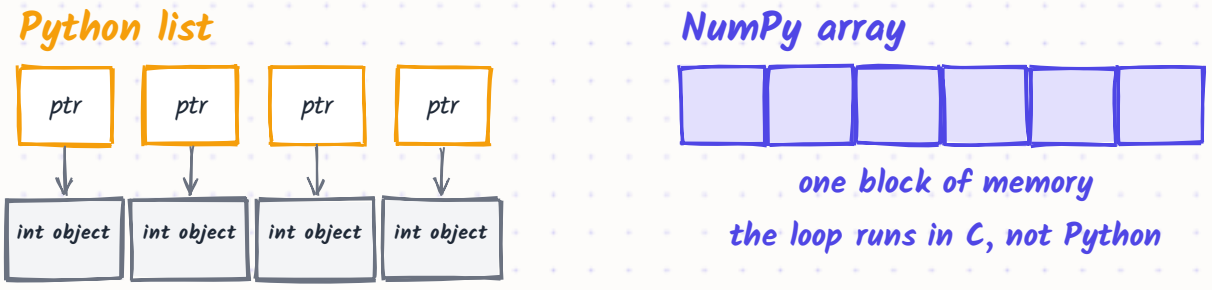
Lesson 3 · Lab 1 · why loops are slow

In this lesson → ① Why NumPy ② Arrays ③ Loop vs vectorised ④ Broadcasting ⑤ Masks ⑥ Axis ⑦ 3 mistakes ⑧ Interview Q&A

Lab data: a 1M-row array, timed two ways — everything in this lab runs in a notebook you can copy.

1 Why NumPy is the floor of the stack

> A **NumPy array** is one contiguous block of same-typed numbers. pandas is NumPy with labels, scikit-learn takes arrays, PyTorch copied the API.




```
import numpy as np

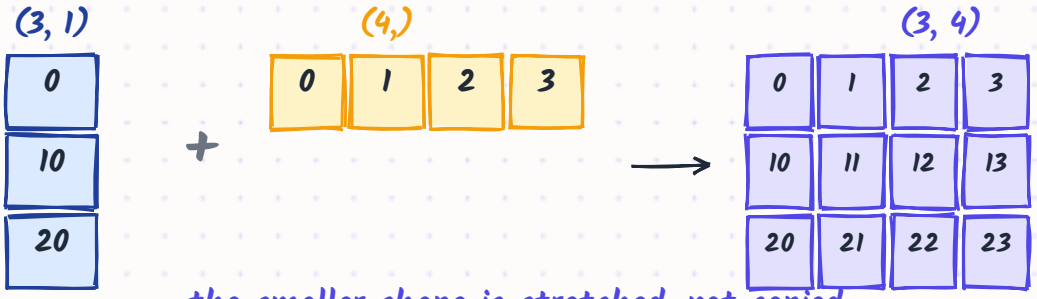
a = np.arange(1_000_000)
a.shape    # (1000000,)
a.dtype    # int64
a.nbytes/1e6  # 8.0 MB
```

2 The whole point: vectorise

Way	Code	Time (1M values)
Python loop	`for x in a: total += x*x`	~124 ms
NumPy	`np.dot(a, a)`	~1.1 ms

Rule
Writing a **for loop over data**? There is almost always an array operation for it.

3 Broadcasting



the smaller shape is stretched, not copied

→ Line the shapes up **from the right**

→ Each pair must be equal, or one of them must be 1

→ The size-1 dimension is stretched — **no copy is made**

```
x = np.arange(3).reshape(3, 1)  # (3, 1)
y = np.arange(4)                # (4,)
(x * 10 + y).shape              # (3, 4)

# real use: centre every column
X - X.mean(axis=0)
```

4 Selecting: slices and masks

```
c = np.arange(10)
c[2:5]              # [2 3 4]

mask = c % 2 == 0
c[mask]             # [0 2 4 6 8]
c[(c > 3) & (c < 8)]   # [4 5 6 7]  brackets matter
```

Read it aloud

`c[c % 2 == 0]` is "c, where c is even". Same idea filters a DataFrame next lesson.

5 Aggregations and the axis trap

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])

b.sum(axis=0)  # [5 7 9]
b.sum(axis=1)  # [6 15]
```

Remember
axis = the dimension that disappears.
(2,3) with axis=0 → 3 numbers.
(2,3) with axis=1 → 2 numbers.

→ `sum · mean · min · max · std · argmax` all take `axis`

→ `np.where(cond, a, b)` — vectorised if/else

→ `np.clip, np.round, np.unique(x, return_counts=True)`

6 3 mistakes that will bite you

Mistake	What happens	Fix
A slice is a view	editing the slice edits the original	`a[:5].copy()`
dtype overflow	`int8 [1,2,3] * 100 → [100, -56, 44]`, no error	check `.dtype`, use int64/float64
`np.append` in a loop	copies the whole array each time — $O(n^2)$	build a list then convert, or `np.zeros(n)`

Cheat sheet — the calls you will actually use

Make	Reshape	Compute
`np.array(list)`	`a.reshape(3, -1)`	`a.sum(axis=0)`
`np.arange(n)`	`a.T`	`a.mean() / a.std()`
`np.zeros((r, c))`	`a.ravel()`	`a.argmax()`
`np.linspace(0, 1, 11)`	`np.concatenate([a, b])`	`np.where(cond, x, y)`

7 Interview corner

Q. Why is NumPy faster than a Python list?

A. One contiguous block of memory, a single dtype, and the loop runs in compiled C — no pointer chasing, no unboxing per element.

Q. What is broadcasting?

A. Shapes are aligned from the right; each pair must match or be 1. The size-1 dimension is stretched without copying.

Q. View or copy?

A. Basic slicing gives a view (shares memory). Fancy indexing and boolean masks give a copy.

Q. What does axis=0 mean?

A. Collapse along rows — the row dimension disappears, so you get one value per column.

⚡ Revise in 60 seconds

- → arrays, not loops — vectorise everything
- → contiguous memory + one dtype + C loop = the speed
- → broadcasting: align from the right, 1 stretches
- → masks: `c[(c>3) & (c<8)]`, brackets required
- → axis = the dimension that disappears
- → traps: views, dtype overflow, append in a loop

Lab checklist

- ✓ ran both timings and saw the gap yourself
- ✓ printed `.shape`, `.dtype`, `.nbytes` on a real array
- ✓ filtered an array with a boolean mask
- ✓ summed with `axis=0` and `axis=1` and can explain the difference
- ✓ reproduced the view-vs-copy surprise once, on purpose

Next lesson → DS-04
Lab: **pandas** — the 12 calls that cover 90% of the work
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**